PUBLIC

Royco Dawn — MIESC audit triage

Target: roycoprotocol/royco-dawn @ d5f9360c

Scope: Cantina competition — first-party src/ (54 files, ~7.5k LoC)

Scan: 0 Critical · 1 High · 41 Medium · 53 Low · 287 Info

Verdict: the one scanner High is a false positive, in one of the competition's two reward classes; the contest closed with 262 submissions, 0 valid, 0 High

MIESC — smart-contract security audit tooling · dogfooding case study. Triage layer over a default static scan. Public information from public repositories only; static analysis only; no exploitation; nothing submitted to any program.

Royco Dawn (royco-dawn) — MIESC audit triage memo

Target: `roycoprotocol/royco-dawn` @ `d5f9360c` · Cantina competition scope (first-party `src/`, 54 files / ~7.5k LoC) **What it is: Royco (v2) Dawn**, a *perpetual risk-tranching* protocol. It splits an underlying yield opportunity into a **senior** tranche (protected, up to a market-defined drawdown, by the junior as first-loss capital) and a **junior** tranche (leveraged: absorbs first loss, receives a share of senior yield on top of its own). Contract flow is **tranche → kernel → accountant → YDM** and back: ERC-7540 async-vault tranches (`RoycoVaultTranche`, senior/junior), a per-market `RoycoKernel` that executes deposits/redemptions against external venues (ERC4626, Aave v3), a `RoycoAccountant` that tracks each tranche's NAV / coverage / fees, and a **Yield Distribution Mechanism** (`AdaptiveCurveYDM` / `StaticCurveYDM`) that continuously re-prices the senior/junior yield split from supply-demand. Oracle quoters price identical-asset pairs. Access is OpenZeppelin `AccessManaged` (`restricted`) plus the ERC-7540 operator model. **Toolchain:** Foundry / **solc 0.8.28** (evm *prague*, via-IR, optimizer 3500) + **Slither 0.11.3**, reusing forge build-info. **Scan (contest scope = `src/`): 382 owned findings — 0 Critical / 1 High / 41 Medium / 53 Low / 287 Info**. **Method note:** MIESC ran Slither with its **default detector set**, deliberately *without* the project's `foundry.toml [lint]` config, over the **whole first-party** `src/` (the contest's stated scope). 245 dependency results under `lib/` were excluded — including **11** `lib/` **Highs, all solady assembly-math FPs** (7 `incorrect-shift` + 4 `incorrect-exp`); **none was an** `arbitrary-send-*`.

> Triage layer. **Nothing here is a submission.** No PoC was built or submitted. The Cantina competition on this commit is **complete** (Royco-team judged; the competition report is public), so this write-up carries no live-competition embargo; it reconciles MIESC's output against the project's **own published reports** (Cantina competition report + Hexens + Certora FV, all committed to the repo). Any disclosure is the user's own action.

Headline

The scanner's one High is a false positive — and it lands in one of the two classes the competition rewarded, where 262 submissions produced zero valid bugs. Royco limited the Cantina reward scope to two high-severity classes: "*funds sent to non-whitelisted addresses, or to addresses not specified by whitelisted addresses*" and "*funds permanently locked or unrecoverable.*" The first is the **arbitrary-recipient transfer** class that Slither's `arbitrary-send-erc20` nominally detects. The Royco team's published verdict: **"There were no valid findings submitted that fulfilled the requirements"** — 262 submissions, **0 valid, 0 High**. MIESC produced exactly **one** finding in that class (`RoycoVaultTranche.requestDeposit`), and triage confirms it is a false positive for the same structural reason the humans found nothing: the recipient is the protocol's own fixed kernel, and the source is double-gated by the AccessManager and the ERC-7540 operator model. **Eighth target in a row, and still 0 exploitable Highs from the static scan.** The real residual risk in Dawn — found by its deep manual review and formal verification — lives in **tranche NAV / fee-accounting invariants**; and for the one real issue that *does* sit behind a Slither flag (an ERC-777 reentrancy in `stRedeem`), the scanner surfaced the site but cannot adjudicate it — which is exactly the job of the triage layer.

High findings — per-site verdict (1/1 FP)

`arbitrary-send-erc20` → **FP** · `RoycoVaultTranche.requestDeposit()` **L301-327**

```
function requestDeposit(TRANCHE_UNIT _assets, address _controller, address _owner)
    external virtual override(IRoycoAsyncVault)
    restricted                      // OZ AccessManaged authority check
    whenNotPaused
    onlyCallerOrOperator(_owner)    // ERC-7540 operator model
    executionIsAsync(Action.DEPOSIT)
    returns (uint256 requestId, bytes memory metadata)
{
    address kernel_ = kernel();
    IERC20(asset()).safeTransferFrom(_owner, kernel_, toUint256(_assets));   // flagged
    ...
}
```

Slither flags the `safeTransferFrom(_owner, kernel_, …)` because `_owner` is a function parameter (an "arbitrary from"). Reading the guards, it is not attacker-controllable into a theft:

1. **Source is double-gated.** `onlyCallerOrOperator(_owner)` calls `_isCallerOrOperator` (L781), which is `msg.sender == _owner || isOperator[_owner][msg.sender]` — the canonical **ERC-7540** operator-approval invariant: you can only pull `_owner`'s assets if you *are* `_owner` or `_owner` approved you as operator via `setOperator`. On top of that, `restricted` (OZ `AccessManagedUpgradeable`, via `RoycoAuth`) requires the caller to be permissioned by the market's AccessManager authority. Two independent gates, not one.
2. **Destination is fixed, not arbitrary.** The funds go to `kernel_ = kernel()`, the tranche's own kernel address read from storage (set at initialization), **not** to any caller-supplied address. There is no "arbitrary recipient" here at all — which is precisely the property the competition's reward scope was about.
3. **Canonical shape.** This is the standard ERC-7540 `requestDeposit(assets, controller, owner)` pull: owner's assets move into the vault/kernel to be queued. `arbitrary-send-erc20` fires on the parameterized `from` and cannot model the operator-approval or AccessManager invariant that authorizes it.

FP. The contest's own rules reinforce this: the Royco scope treats **whitelisted parties as trusted** ("assumed trusted, funds recoverable") and excludes **automated-analyzer output** ("Automated findings by Lightchaser" are out of scope) — so a static `arbitrary-send` hit on a permissioned, fixed-destination pull is non-rewardable *by the competition's own definition*, independent of my reading.

Where the static lens ends — Dawn's real residual risk

The scanner's single High is noise. The issues the protocol's **deep reviews** actually surfaced are **tranche-accounting and economic invariants**. From the repo's own committed reports (Hexens, Certora FV) — characterized here at the **class** level, since those reviews were performed on different commits than the contest commit, not claimed as live in `d5f9360c`:

- **NAV division-by-zero** — "JT may be unable to redeem their shares due to division by zero" when effective NAV reaches a boundary state.
- **Fee-threshold unmintability** — "Accrued JT protocol fees become unmintable when effective NAV reaches [a threshold]."
- **Fee-share compounding** — "Tranche fee recipient's shares compound into higher fee rates."
- **ERC-777-hook reentrancy in `stRedeem`** — exploitable *only if* the senior underlying is an ERC-777 token whose transfer callback (inside `_withdrawAssets`) lets an attacker reenter and re-run `_preOpSyncTrancheAccounting`, corrupting `stEffectiveNAV` / `stRawNAV`. (Hexens labels the finding under `RoycoAccountant.sol`; the `stRedeem` entrypoint itself lives in `RoycoKernel.sol` and calls into the accountant's sync functions.)

The first three have **no static flag at all** — they are pure accounting/economic invariants (division-safety at a NAV boundary, a fee-mint threshold, fee-share compounding) that a pattern detector has no vocabulary for. The honest and more interesting case is the fourth, because it is **not** a blind spot on location. Slither flagged **`stRedeem` itself — the exact function the finding turns on — as `reentrancy-no-eth` (Medium)**, alongside `stDeposit`, `jtDeposit` and `jtRedeem` in `RoycoKernel.sol` and `deposit` / `redeem` / `requestRedeem` / `claimCancelRedeemRequest` in the tranche: **eight `reentrancy-no-eth` Mediums** across the deposit/redeem accounting surface (the other eleven `-no-eth` Mediums are `restricted` admin setters, oracle-quoter `setConversionRate`, and `deployMarket` — config, not value flow). The static scan put a Medium-severity reentrancy flag squarely on the right site.

What it **cannot** do is adjudicate that set. Those eight flags are overwhelmingly benign: the external call is to the protocol's own trusted kernel / accountant / venue under an ordering that is safe for a standard ERC-20, which is why 262 submissions and the professional reviews did not treat them as a drain. Exactly **one** of them becomes a real Medium — and only under a precondition Slither never assesses (that the senior underlying is an ERC-777 whose transfer hook fires inside `_withdrawAssets`) with a consequence it cannot model (the reentrant `_preOpSyncTrancheAccounting` corrupts the tranche NAV invariant). Slither ranks all eight identically; it has no way to know which token is ERC-777, and no model of the NAV-solvency invariant that the reentry breaks. So the lesson is sharper than "static misses semantic bugs": **the static layer produces the candidate set, but cannot rank or confirm within it.** The discrimination — one real ERC-777 vector out of eight look-alike reentrancy flags — is precisely the work the triage and semantic-review layer exists to do, and it rests on a token-standard precondition and an accounting invariant that live entirely outside the detector's model.

Other classes (characterized)

Class	Count (src)	Verdict
`naming-convention`	260	Style; the project itself disables mixed-case lints in `foundry.toml`. Noise.
`reentrancy-no-eth` / `-events` / `-benign`	19 / 28 / 14	The 19 `-no-eth` Mediums split between 11 `restricted` admin setters / quoter `setConversionRate` / `deployMarket` (config, not drainable) and the eight deposit/redeem accounting entrypoints (`stRedeem` / `stDeposit` / `jtDeposit` / `jtRedeem` / `deposit` / `redeem` / `requestRedeem` / `claimCancelRedeemRequest`) discussed above — the candidate set the triage layer must adjudicate, not dismiss.
`unused-return`	12	Ignored return codes on trusted internal/kernel calls; standard, characterized by class.
`dead-code`	13	Unused internal helpers / interface stubs; non-security.
`uninitialized-local`	5	Zero-seeded accumulators/structs written before read; benign, characterized by class.
`incorrect-equality`	4	`== 0` / sentinel guards in NAV/units math; characterized by expression, not individually opened.
`divide-before-multiply`	1	One mantissa-order expression in the YDM/units math; fixed-point truncation idiom.
`timestamp` (9), `assembly` (8), `low-level-calls` (3), `shadowing-local` (2), `pragma` / `solc-version` / `cyclomatic-complexity`	—	Noise / standard; characterized by detector class, not individually opened.

Reconciliation against the project's published reports

Three public baselines, all committed to `royco-dawn`:

1. **Cantina competition report** (`audit/Cantina-Royco-Dawn-Whitelist.pdf`). Royco-team judged; reward scope limited to two classes — arbitrary-/non-whitelisted-recipient fund flows and permanently-locked funds. Verdict: **"There were no valid findings that fulfilled the requirements"** — 262 submissions, **0 valid, 0 High**. MIESC's single High is in the first of those two classes and is an FP — a **true negative on the reward class**, concordant with 262 auditors.
2. **Hexens** (`audit/Hexens-Royco-Dawn-{1,2,3}.pdf` , `…-Whitelist.pdf`). Deep manual review; the representative report found **3 Medium, 2 Low, 8 Info — no High/Critical**, all in the NAV/fee-accounting classes above. **None is a class Slither produces.**
3. **Certora** (`formal-verification/Certora-Royco-Dawn-FV.pdf`). Formal verification of tranche invariants — the method appropriate to exactly the properties the static layer cannot express.

MIESC's static scan surfaced **zero** of the judged/real issues and reconfirmed its one raw High as a false positive. That is not a conflict with the reviews; it is the honest shape of the tool — the detectors fire where a pattern matches (a permissioned, fixed-destination pull), and the real risk sits in invariants (NAV solvency, fee accounting, token-callback reentrancy) that only semantic review and FV reach.

Bottom line

- **No submittable bug** from static scan + triage — **eighth** target in a row (Lombard, Doppler, Valantis STEX, YieldNest, Arcadia, v4lend, Malda, **Royco Dawn**), all raw Highs being false positives.
- **The one scanner High is in one of the competition's two reward classes and is still an FP** — a `requestDeposit` pull that is double-gated (AccessManager `restricted` + ERC-7540 operator model) and sends to the fixed kernel, never an arbitrary recipient. The Cantina competition closed with **262 submissions, 0 valid, 0 High**; MIESC agrees.
- **The real residual risk is mostly out of the static lens by construction, and where it isn't, the lens can locate but not adjudicate:** NAV division-by-zero, fee-threshold unmintability and fee-share compounding have no static flag at all (pure accounting/economic invariants); the ERC-777-hook reentrancy *does* sit behind a flag — Slither marked `stRedeem` and seven sibling deposit/redeem functions as `reentrancy-no-eth` (Medium) — but the scanner cannot tell the one real ERC-777 vector from the benign majority, because that rests on a token-standard precondition and a NAV-solvency invariant it does not model. The discrimination is the triage layer's job; the protocol's manual review (Hexens) and formal verification (Certora) were the right instruments for the rest.
- **Honest limits:** this is the **public contest commit** after **262 prior submissions** plus multiple professional reviews, so anything the scanner surfaces is almost certainly already known; verdicts are from reading the code against the NatSpec and the project's public reports; **no PoC was built or run**. MIESC's Slither lens does not cover the NAV/accounting/economic semantic classes the real issues occupy.

Appendix — Evidence trail

- Normalized scan: `results.json` (owned = first-party `src/` ; `lib/` dependencies excluded — the 11 excluded `lib/` Highs are all solady assembly-math FPs, `incorrect-shift` / `incorrect-exp` , no `arbitrary-send`).
- Reports: `triage-focused.pdf` (focused triage); the full premium PDF is regenerable from `results.json` and is not committed.
- Toolchain: HTTPS clone @ `d5f9360c` + submodules → `forge build --root <repo> --build-info --skip test/script` (solc 0.8.28, evm prague, via-IR) → `slither <repo> --foundry-ignore-compile --json` (Slither 0.11.3, default detectors). Build/scan were invoked from a safe working directory with the repo passed by absolute path (never `cd` -ing into the untrusted clone); Slither's `filename_relative` paths carry a `royco-audit/repo/` prefix, which the normalizer strips before scope-matching.

- Target pin: `roycoprotocol/royco-dawn` @ `d5f9360cf8c2d49571b16c311ba86b085e5acd8e` (Cantina "Royco Dawn" competition, 20-27 Jan 2026).
- Only **public source from the public GitHub repository** and the project's **own public audit reports** were accessed; **static analysis only**, no live systems / RPC / credentials, no exploitation, nothing submitted anywhere.